

AI Agents & der Thought-Action-Observation-Zyklus

Applied AI · Master Informatik · HTWG Konstanz · SS26

HF Agents Course – Unit 1 & 2, Yao et al. (2022)

Lernziele dieser Vorlesung

- den Begriff **AI Agent** präzise definieren und vom reinen LLM-Aufruf abgrenzen
- den **Thought-Action-Observation-Zyklus** als Code-Schleife lesen und beschreiben
- die drei Thought-Arten (**Planning, Analysis, Self-Reflection**) mit Beispielen belegen
- **Chain-of-Thought** (CoT) und **ReAct** vergleichen und situationsgerecht einsetzen
- die Action-Formate **JSON Agent**, **Code Agent** und **Function-Calling** unterscheiden
- das **Stop-and-Parse-Prinzip** erklären und dessen Bedeutung einschätzen
- beschreiben, wie Observations in den Kontext zurückfließen
- den Aufbau eines **System Prompts** mit Tool-Beschreibungen skizzieren

Einführung: Was ist ein AI Agent?

In VL 01 haben wir gesehen, was ein LLM ist und was es **nicht** kann: es kann keine echten Aktionen ausführen, keine aktuellen Daten abrufen, sich nicht an frühere Gespräche erinnern. Genau hier setzt der AI Agent an.

■ Definition: AI Agent

Ein **AI Agent** ist ein System, das:

1. eine Aufgabe (Goal) empfängt,
2. über Werkzeuge (**Tools**) mit seiner Umgebung interagieren kann,
3. in einem autonomen Zyklus aus **Denken, Handeln** und **Beobachten** operiert.

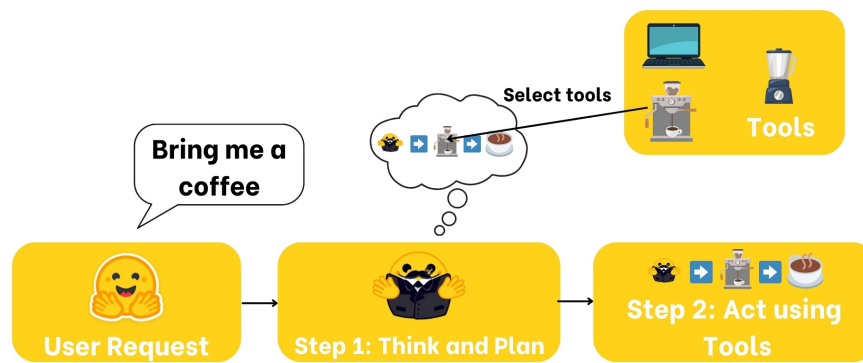
Der entscheidende Unterschied zum LLM-Direktaufruf: Der Agent kann **mehrere Schritte** ausführen, Zwischenergebnisse verarbeiten und seinen Plan dynamisch anpassen.

Analogie

Ein LLM allein ist wie ein brillanter Berater im Büro ohne Telefon. Ein **Agent** gibt ihm ein Telefon, einen Computer und die Erlaubnis, selbst anzurufen und Nachforschungen anzustellen – er wird vom Denker zum **Handelnden**.

LLM vs. Agent

Das LLM ist der **Reasoning-Kern** des Agenten – es wird nicht ersetzt, sondern eingebettet. Der Agent-Rahmen (Framework) gibt ihm Werkzeuge und Gedächtnis.



6

Figure 1: **Abb.:** Agenten-Prozess: Goal → Thought → Action → Observation (HF Agents Course Unit 1)

Merkmal	LLM (standalone)	AI Agent
Ausführung	Einmaliger Aufruf	Mehrere Schritte (Loop)
Werkzeuge	Keine	Beliebige Tools
Zustand	Kein persistenter Zustand	Memory / History
Anpassung	Statisch	Dynamisch (re-plan)
Autonomie	Gering	Hoch

● Wetterabfrage: LLM vs. Agent

LLM allein:

User: "Wie ist das Wetter in Berlin?"
 LLM: "Das Wetter in Berlin ist typischerweise..."
 [Trainingsdaten -- keine Echtzeit-Daten]

Agent:

User: "Wie ist das Wetter in Berlin?"
 Thought: "Ich brauche Echtzeit-Daten -- get_weather aufrufen."
 Action: get_weather("Berlin")
 Obs: "15 Grad, bewölkt"
 Answer: "Heute in Berlin: 15 Grad, bewölkt."

Der Thought-Action-Observation-Zyklus

Der Zyklus ist das **Herzstück** jedes Agenten. Alles, was ein Agent tut, lässt sich auf diesen dreiteiligen Kreislauf zurückführen – und fast alle Fehler lassen sich auf genau einen der drei Schritte zurückführen.

Analogie

Denk an einen erfahrenen Detektiv: Er **überlegt** (Thought), welche Spur er verfolgt, **handelt**

(Action – z.B. Zeuge befragen), und wertet das **Ergebnis aus** (Observation). Dann beginnt der nächste Zyklus – bis der Fall gelöst ist.

Der Kern ist buchstäblich eine while-Schleife im Code:

```
while not done:
    thought = llm.think(context)           # Was ist der nächste Schritt?
    action  = llm.choose_action(thought, tools) # Welches Tool, welche Parameter?
    result  = execute_tool(action)         # Tool wird AUSSERHALB des LLM
    ausgeführt
    context += observation(result)         # Ergebnis zurückfließen lassen
```

Bild (online): Thought → Action → Observation Zyklus (HF Agents Course Unit 1)

Figure 2: **Abb.:** Thought → Action → Observation Zyklus (HF Agents Course Unit 1)

★ Kernaussage

Das LLM **denkt und plant**, aber es **handelt nicht selbst**. Der Agent-Rahmen (Framework) führt die Tools aus und gibt die Ergebnisse zurück. Das LLM sieht bei jedem Schritt nur Text.

Tiefe: Thought – Der innere Monolog

Was ist ein Thought?

■ Thought = Innerer Monolog des Agenten

Ein **Thought** ist der interne Planungs- und Entscheidungsschritt des Agenten. Das LLM analysiert den aktuellen Kontext – bisherige Nachrichten, Observations, verfügbare Tools – und bestimmt, was als nächstes zu tun ist.

Der Thought ist für den Endnutzer normalerweise **nicht sichtbar**, wird aber in Logs und Debug-Outputs angezeigt.

Es gibt verschiedene Typen von Thoughts – je nach Situation nutzt das Modell unterschiedliche Strategien:

Thought-Typ	Beispiel
Planning	“Ich teile die Aufgabe in 3 Schritte: 1) Daten sammeln, 2) analysieren, 3) Bericht schreiben.”
Analysis	“Die Fehlermeldung zeigt ein Problem bei den DB-Verbindungsparametern – ich prüfe den Connection String.”
Decision Making	“Angesichts des Budgets empfehle ich die mittlere Option, da sie das beste Preis-Leistungs-Verhältnis bietet.”
Self-Reflection	“Mein letzter Ansatz hat nicht funktioniert – ich versuche eine andere Strategie und nutze diesmal search statt lookup.”

Chain-of-Thought (CoT) vs. ReAct

Zwei zentrale Paradigmen für das Reasoning im Agenten-Kontext:

Analogie

Chain-of-Thought ist wie ein Schüler, der bei einer Matheaufgabe den Rechenweg aufschreibt – alles passiert im Kopf, ohne externe Hilfsmittel. **ReAct** ist wie derselbe Schüler, der Taschenrechner, Internet und Bibliothekskatalog nutzen darf.

Chain-of-Thought (CoT) ist eine Prompting-Technik, die das Modell zum schrittweisen Denken bringt – **ohne** externe Tools:

Frage: Wie viel ist 15% von 200?

Thought: 10% von 200 = 20. 5% von 200 = 10. Also: 15% = 30.

Antwort: 30

Trigger-Satz: *“Let’s think step by step.”*

ReAct (Yao et al., 2022) kombiniert internes Reasoning mit externen Aktionen. Das LLM wechselt zwischen Denken (Thought) und Handeln (Action), bis die Antwort gefunden ist:

(a) Few-shot	(b) Few-shot-CoT
Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now? A: The answer is 11. Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there? A: (Output) The answer is 8. ✗	Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now? A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11. Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there? A: (Output) The juggler can juggle 16 balls. Half of the balls are golf balls. So there are $16 / 2 = 8$ golf balls. Half of the golf balls are blue. So there are $8 / 2 = 4$ blue golf balls. The answer is 4. ✓
(c) Zero-shot	(d) Zero-shot-CoT (Ours)
Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there? A: The answer (arabic numerals) is (Output) 8 ✗	Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there? A: Let’s think step by step. (Output) There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls. ✓

Figure 3: **Abb.:** Chain-of-Thought vs. ReAct -- CoT nutzt rein internes Reasoning, ReAct ergänzt externes Handeln (HF Agents Course Unit 1)

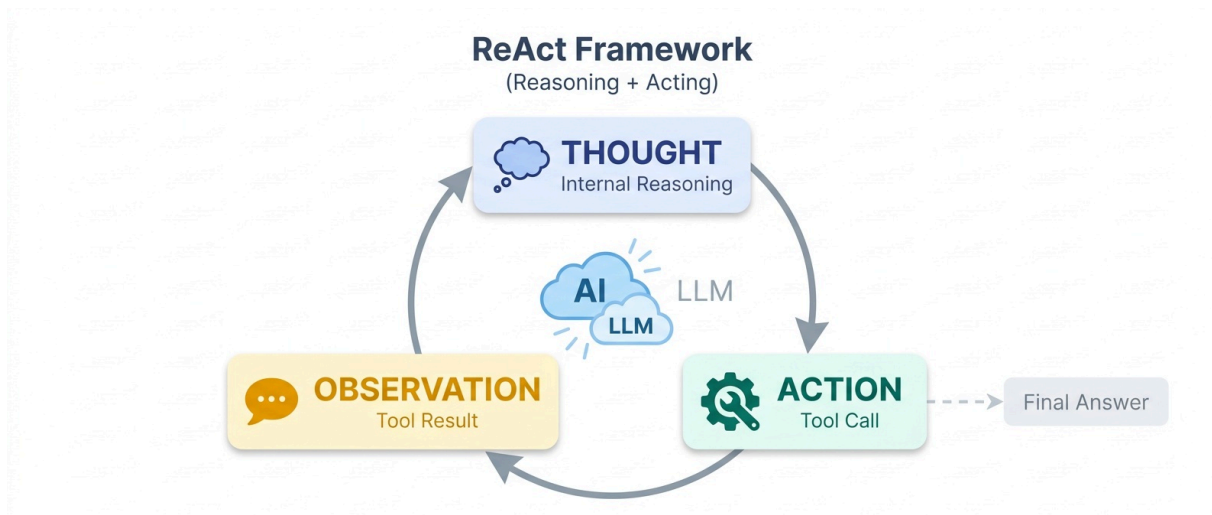


Figure 4: **Abb.:** Der ReAct-Zyklus: Thought (internes Reasoning), Action (Tool-Aufruf) und Observation (Tool-Ergebnis) im Wechsel (PaperBanana-Diagramm)

Thought: Ich muss das aktuelle Wetter in Paris finden.
 Action: Search["weather in Paris"]
 Obs: 18°C, bewölkt.
 Thought: Jetzt kenne ich das Wetter. Ich kann antworten.
 Action: Finish["18°C, bewölkt in Paris."]

Merkmal	Chain-of-Thought	ReAct
Schrittweise Logik	✓	✓
Externe Tools	✗	✓
Geeignet für	Logik, Mathe, Schlussfolgerungen	Informationssuche, dynamische Aufgaben

⚠ Achtung

Neuere Modelle (DeepSeek-R1, OpenAI o1) nutzen strukturierte `<think>...</think>` Tokens. Das ist eine **Training-Level-Technik** – das Modell wurde darauf trainiert, intern zu denken, bevor es antwortet. Kein reines Prompting, sondern eingebettetes Verhalten.

Tiefe: Action – Handeln in der Umgebung

Nach dem Thought kommt die Action. Das LLM hat entschieden, was zu tun ist – nun muss es diesen Entschluss in einem Format kommunizieren, das der Agent-Rahmen maschinell lesen und ausführen kann.

■ Action = Textuelle Beschreibung eines Tool-Aufrufs

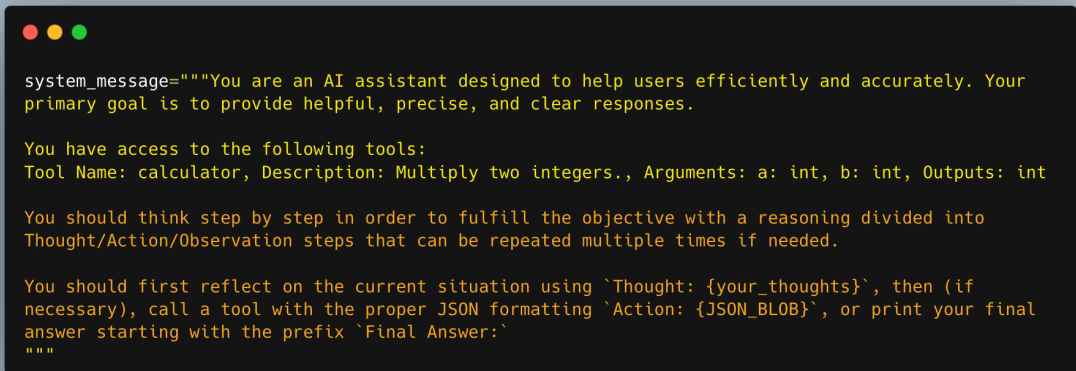
Eine **Action** ist der konkrete Schritt, den ein Agent unternimmt – z.B. eine Web-Suche starten, Code ausführen oder eine API aufrufen.

Wichtig: Das LLM **erzeugt nur Text**, der die Aktion beschreibt. Der **Agent-Rahmen** führt die Aktion tatsächlich aus. Das LLM selbst hat keinen direkten Zugriff auf die Außenwelt.

Das Stop-and-Parse-Prinzip

★ Kernaussage

Das **Stop-and-Parse-Prinzip** ist der Mechanismus, der LLM-Outputs strukturiert und maschinell verarbeitbar macht: Das LLM stoppt nach der Action, der Framework-Layer parst sie und führt sie aus. Ohne explizites Stoppen würde das LLM weiterschreiben – und die Action “verunreinigen”.



```
system_message="""You are an AI assistant designed to help users efficiently and accurately. Your primary goal is to provide helpful, precise, and clear responses.

You have access to the following tools:
Tool Name: calculator, Description: Multiply two integers., Arguments: a: int, b: int, Outputs: int

You should think step by step in order to fulfill the objective with a reasoning divided into Thought/Action/Observation steps that can be repeated multiple times if needed.

You should first reflect on the current situation using `Thought: {your_thoughts}`, then (if necessary), call a tool with the proper JSON formatting `Action: {JSON_BLOB}`, or print your final answer starting with the prefix `Final Answer:`
"""
```

Figure 5: **Abb.:** System-Prompt-Zyklus: Wie Tool-Aufrufe in den Agenten-Loop integriert werden (HF Agents Course Unit 1)

Thought: Ich muss das aktuelle Wetter in New York prüfen.

Action:

```
{
  "action": "get_weather",
  "action_input": {"location": "New York"}
}
```

Nach dem schließenden } stoppt das LLM. Der Framework-Layer parst das JSON und ruft `get_weather(location="New York")` auf – und gibt das Ergebnis als Observation zurück.

Agent-Typen nach Action-Format

Es gibt verschiedene Standards, wie eine Action kodiert wird. Sie unterscheiden sich in Ausdrucksstärke, Robustheit und Sicherheit:

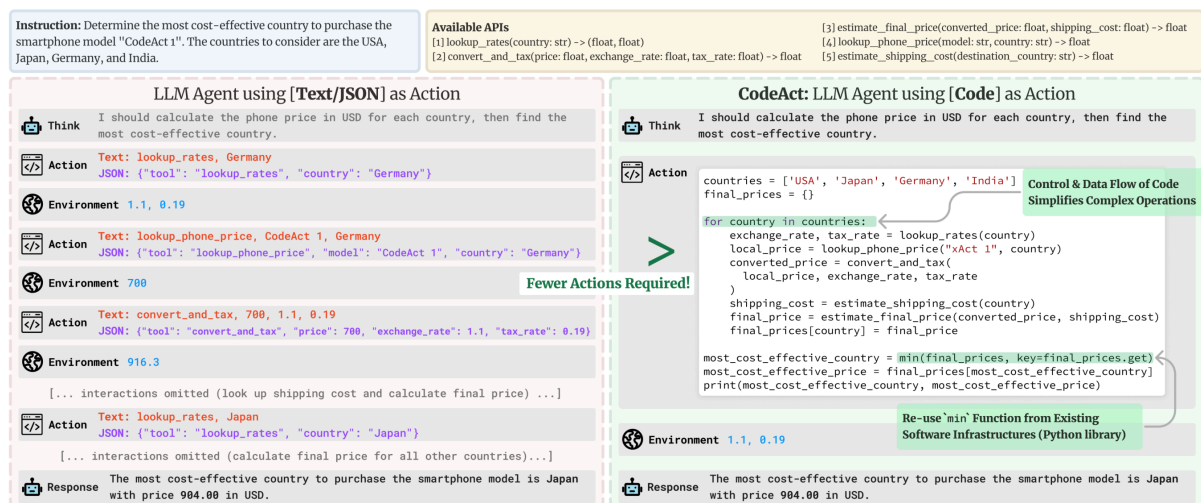


Figure 6: **Abb.:** Code Actions vs. JSON Actions -- Vergleich der Action-Formate (HF Agents Course / smolagents Docs)

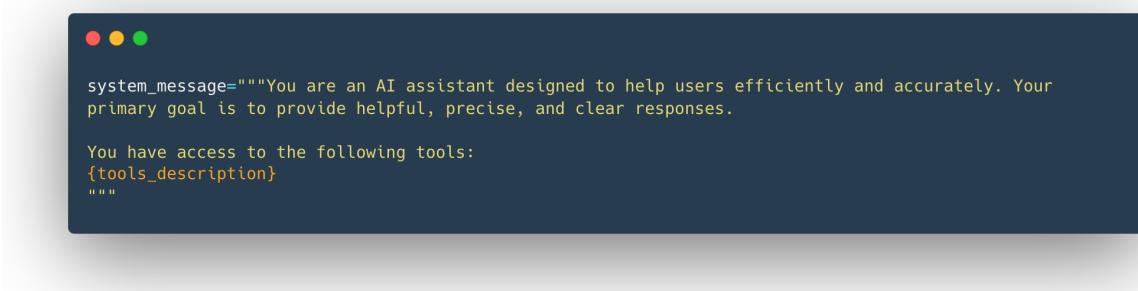


Figure 7: **Abb.:** System-Prompt mit Tool-Beschreibungen -- das LLM erhält Tools als Text (HF Agents Course Unit 1)

Agent-Typ	Beschreibung
JSON Agent	Action als JSON-Objekt – einfach zu parsen, weniger ausdrucksstark, Standard in vielen Frameworks
Code Agent	Agent generiert ausführbaren Python-Code – mächtiger (Schleifen, Logik), aber größere Angriffsfläche
Function-Calling Agent	Unterart des JSON Agent – für jede Action eine eigene Message (OpenAI-API-Standard, native Unterstützung in vielen Modellen)

Code Agents: Mehr Macht, mehr Verantwortung

Ein **Code Agent** generiert statt JSON einen ausführbaren Python-Block. Das ermöglicht komplexe Logik direkt in der Action:

```
def get_weather(city: str) -> str:
    import requests
    response = requests.get(f"https://api.weather.com/v1/location/{city}")
    if response.status_code == 200:
        return response.json().get("weather", "No data")
    return "Error: Unable to fetch weather data."

result = get_weather("New York")
```

```
final_answer = f"Current weather in New York: {result}"  
print(final_answer)
```

Vorteile gegenüber JSON: Direktere Integration, bessere Debuggbarkeit, Schleifen und Conditionals im Action-Step möglich, kein separater Parser nötig.

⚠ **Sicherheitshinweis: Code Agents**

LLM-generierten Code auszuführen birgt erhebliche Risiken. Ein Angreifer könnte Schadcode über Tool-Ausgaben einschleusen (**Prompt Injection**). Frameworks wie smolagents integrieren Sandboxing (Docker, E2B Sandbox). **Immer in einer isolierten Sandbox ausführen – niemals direkt auf dem Host-System!**

Tiefe: Observation – Feedback integrieren

Das Ergebnis einer Aktion ist nur dann nützlich, wenn es korrekt in den Kontext zurückfließt. Die Observation ist das Bindeglied, das dem LLM ermöglicht, aus seinen Aktionen zu “lernen” – innerhalb eines einzelnen Runs. Ohne diesen Rückkanal wäre der Agent blind gegenüber dem, was er tut.

■ **Observation = Tool-Ergebnis als neue Kontextnachricht**

Eine **Observation** ist das Feedback nach einer Action. Sie wird als neue Nachricht **am Ende des Kontexts** angehängt – der Agent “sieht” beim nächsten Thought automatisch das Ergebnis der letzten Aktion.

Observation: "Current weather in New York: partly cloudy, 15°C, 60% humidity."

Der genaue Ablauf nach einer Action:

1. Action parsen → Tool-Name und Argumente extrahieren
2. Tool aufrufen → Ergebnis erhalten
3. Ergebnis als **Observation** an den Kontext anhängen
4. LLM plant beim nächsten Thought mit dem erweiterten Kontext weiter

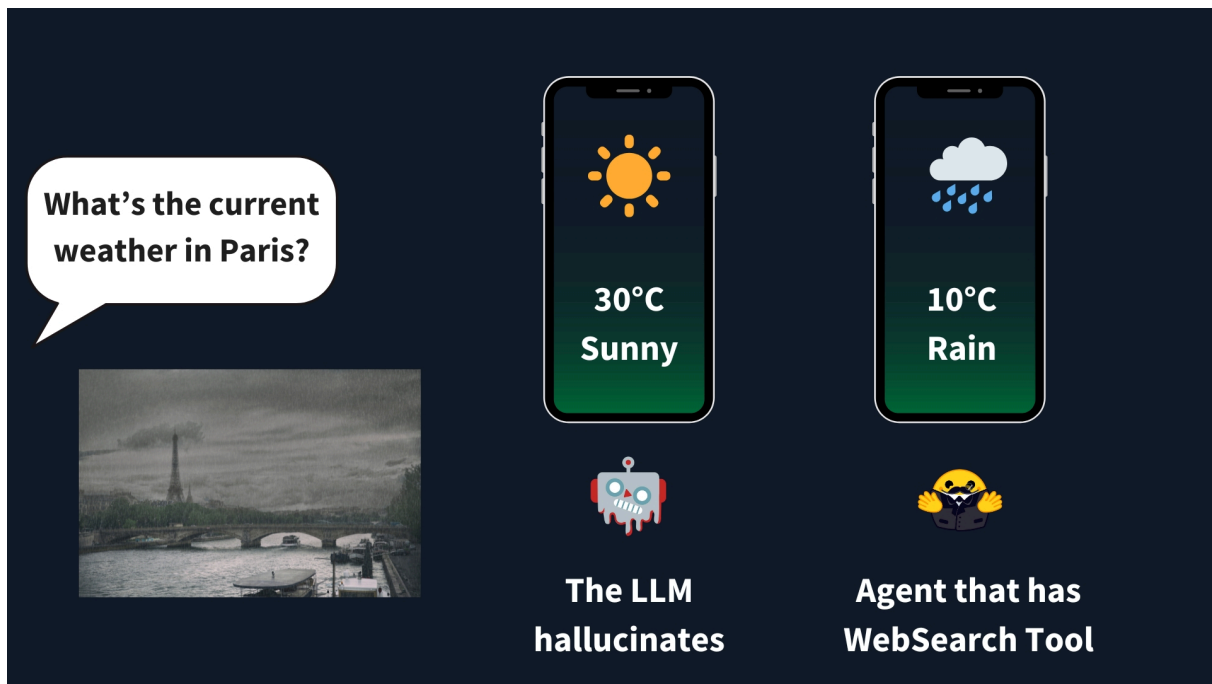


Figure 8: **Abb.:** Beispiel: Wetter-Agent -- kompletter TAO-Zyklus mit Tool-Aufruf (HF Agents Course Unit 1)

- **Vollständiger TAO-Zyklus -- Alfred, der Wetter-Agent**

[System Prompt: Du bist Alfred. Du hast Zugriff auf: `get_weather(location)`]

User: "Wie ist das Wetter in New York?"

Thought: "Der Nutzer fragt nach aktuellem Wetter. Ich habe `get_weather`. Ich rufe es mit `location='New York'` auf."

Action: `{"action": "get_weather", "action_input": {"location": "New York"}}`

Obs: "Partly cloudy, 15°C, 60% humidity."

Thought: "Ich habe die Daten. Ich kann jetzt antworten."

Answer: "In New York ist es aktuell 15°C bei 60% Luftfeuchtigkeit und teilweise bewölkt."

Zwei Thought-Schritte, eine Action, eine Observation – das ist ein vollständiger TAO-Zyklus.

Wie das LLM die Konversation "sieht": Messages & System Prompt

Bisher haben wir Thought, Action und Observation abstrakt beschrieben. Technisch gesehen empfängt das LLM bei **jedem** Aufruf die gesamte bisherige Interaktionshistorie als eine einzige, formatierte Texteingabe.

- **Das LLM kennt nur Text -- kein Dialog, nur eine lange Zeichenkette**

Es gibt intern keinen echten "Dialog". Bei jedem LLM-Aufruf wird die gesamte bisherige

Konversation – alle Thoughts, Actions, Observations – zusammengefügt und als Ganzes übergeben. Das Modell “sieht” alles auf einmal.

```
system_message = {
    "role": "system",
    "content": "You are Alfred, a helpful agent. Always be polite.\n"
               "Available tools: get_weather(location: str) -> str"
}
conversation = [
    {"role": "user",      "content": "Wie ist das Wetter in New York?"},
    {"role": "assistant", "content": "Thought: Ich rufe get_weather auf.\n"
                                     "Action: {\\"action\\": \\"get_weather\\", ...}"},
    {"role": "tool",      "content": "Partly cloudy, 15°C, 60% humidity."},
]
```

Der **System Prompt** enthält bei Agenten:

- alle verfügbaren Tools mit Namen, Beschreibung und Parametern
- Anweisungen zum Action-Format (JSON vs. Code)
- Vorgaben für den Thought-Prozess (z.B. “Think step by step”)

⚠ Achtung

Der System Prompt ist nicht optional – er ist die **Bedienungsanleitung** des Agenten. Ein unklarer oder fehlerhafter System Prompt ist die häufigste Ursache für Fehler in Agenten-Systemen.

Zusammenfassung

Was ist ein AI Agent:

- LLM als Reasoning-Kern + Tools + Memory + Loop
- Unterschied: Agenten führen **mehrere Schritte** aus, planen und passen sich an

Der TAO-Zyklus:

- **Thought** – LLM analysiert Kontext und plant den nächsten Schritt (Planning, Analysis, Decision, Self-Reflection)
- **Action** – LLM erzeugt textuellen Tool-Aufruf; Framework führt ihn aus (Stop-and-Parse)
- **Observation** – Tool-Ergebnis wird an den Kontext angehängt; LLM plant weiter

Reasoning-Paradigmen:

- CoT: schrittweise Logik ohne externe Tools
- ReAct: Thought und Action im Wechsel – Standard für Agenten

Action-Formate:

- JSON Agent – einfach, robust
- Code Agent – mächtig, aber Sandbox nötig
- Function-Calling – nativ in OpenAI-kompatiblen APIs

★ Kernaussage

*Der TAO-Zyklus ist kein Abstraktum – er ist buchstäblich die while-Schleife, die jedes Agenten-Framework intern ausführt. Wenn ein Agent “denkt”, generiert das LLM Text. Wenn er “handelt”, ruft der Python-Code eine Funktion auf. Beides ist Text – unterschiedlich **verwendet**.*

Literatur & Quellen

- Yao et al. (2022): *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv:2210.03629 – <https://arxiv.org/abs/2210.03629>
- HF Agents Course, Unit 1: <https://huggingface.co/learn/agents-course/unit1>
- HF Agents Course, Unit 2 (smolagents): <https://huggingface.co/learn/agents-course/unit2>
- smolagents Dokumentation: <https://huggingface.co/docs/smolagents>